

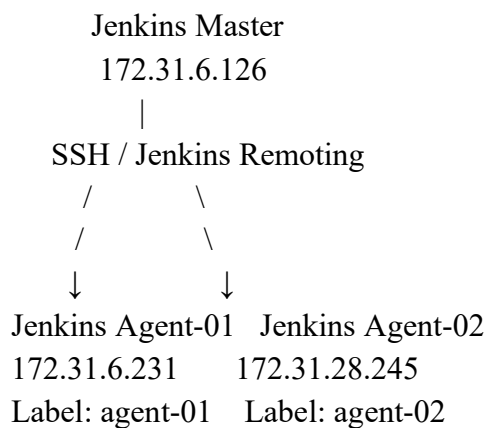
Task 1 — Configure 2 slave machines in the Jenkins master.

Objective

The objective of this task was to configure **two Jenkins Agent machines** and connect them to the Jenkins Master using **SSH-based communication**.

The Jenkins Master is responsible for managing jobs and assigning them to agents. The agents execute the actual build commands.

The final architecture was:



1. Create Jenkins Agent EC2 Instances

Two Amazon EC2 instances were created using **Amazon Linux 2023**.

Agent-01

Name: jenkins-agent-01
Private IP: 172.31.6.231
OS: Amazon Linux 2023
Instance Type: t3.micro

Agent-02

Name: jenkins-agent-02
Private IP: 172.31.28.245
OS: Amazon Linux 2023
Instance Type: t3.micro

Both agents were placed in the same VPC environment as the Jenkins Master so that the Master could communicate with them through their private IP addresses.

Observation

The agents were created as separate EC2 machines. They do not require Jenkins Server installation because the Jenkins Master transfers the required **Remoting agent (remoting.jar)** when the SSH launcher starts the agent.

2. Configure Security Group

SSH communication is required because the Jenkins Master connects to the agents using SSH.

The Agent security groups were configured with:

Inbound Rule

SSH

Protocol: TCP

Port: 22

Source: My IP

A second SSH rule was added to allow communication from the **Jenkins Master security group**:

SSH

Protocol: TCP

Port: 22

Source: Jenkins Master Security Group

Port **8080 was not required on the agents** because Jenkins is running on the Master.

Observation

The Master could communicate with the agents through SSH on port 22.

3. Create Jenkins Linux User on Agent-01

The mentor-required method was followed by creating a dedicated Linux user called jenkins.

On Agent-01:

```
sudo useradd -m -s /bin/bash jenkins
```

The user was verified using:

```
id jenkins
```

Observation

The jenkins Linux account was successfully created.

This allows Jenkins to connect and execute agent processes using a dedicated user instead of directly using ec2-user.

4. Create Jenkins Linux User on Agent-02

The same configuration was performed on Agent-02:

```
sudo useradd -m -s /bin/bash jenkins
```

Then verified:

```
id jenkins
```

Observation

Agent-02 also had a dedicated jenkins Linux user.

5. Install Java on the Agents

Jenkins agents require Java because the Jenkins Remoting agent runs as a Java process.

Java 21 was installed on the agents using:

```
sudo dnf install java-21-amazon-corretto -y
```

The installation was verified:

```
java -version
```

Observation

Java 21 was available on the agent machines.

This was particularly important because Agent-01 initially failed to start with:

```
bash: line 1: java: command not found
```

After installing Java 21, the agent was able to start successfully.

6. Configure SSH Directory on Agent

The SSH directory was created for the jenkins user:

```
sudo mkdir -p /home/jenkins/.ssh
```

Ownership was changed:

```
sudo chown jenkins:jenkins /home/jenkins/.ssh
```

The directory permissions were configured:

```
sudo chmod 700 /home/jenkins/.ssh
```

Observation

The jenkins user now had a secure SSH directory for public-key authentication.

7. Generate SSH Key on Jenkins Master

On the Jenkins Master, the jenkins Linux user was used to create an SSH key specifically for connecting to the agents.

The key was generated using:

```
ssh-keygen -t rsa -b 4096 -m PEM -f ~/.ssh/jenkins_agent_key
```

The important files created were:

~/.ssh/jenkins_agent_key
~/.ssh/jenkins_agent_key.pub

Where:

jenkins_agent_key

↓

Private key

jenkins_agent_key.pub

↓

Public key

Important Security Principle

The **private key remains on the Jenkins Master.**

The public key is placed on the agents.

Master

|

└─ Private Key

jenkins_agent_key

Agent

|

└─ Public Key

authorized_keys

The private key was not copied to the agents.

8. Configure Authorized Keys on Agent-01

The public key from the Jenkins Master was added to:

/home/jenkins/.ssh/authorized_keys

The required ownership was configured:

```
sudo chown jenkins:jenkins /home/jenkins/.ssh/authorized_keys
```

The permissions were configured:

```
sudo chmod 600 /home/jenkins/.ssh/authorized_keys
```

Observation

Agent-01 was configured to accept SSH authentication from the Jenkins Master using the corresponding private key.

9. Configure Authorized Keys on Agent-02

The same Master's public key was added to Agent-02:

```
/home/jenkins/.ssh/authorized_keys
```

Permissions were configured:

```
sudo chown jenkins:jenkins /home/jenkins/.ssh/authorized_keys  
sudo chmod 600 /home/jenkins/.ssh/authorized_keys
```

Observation

Agent-02 was configured to accept SSH authentication from the Jenkins Master.

The same key pair can be used for both agents because each agent has the Master's public key in its `authorized_keys` file.

10. Test SSH Connection to Agent-01

The Jenkins Master was used to test the SSH connection.

From the Master, as the jenkins user:

```
ssh -o IdentitiesOnly=yes -i ~/.ssh/jenkins_agent_key jenkins@172.31.6.231
```

The connection successfully opened an Agent-01 shell:

```
[jenkins@ip-172-31-6-231 ~]$
```

Observation

This confirmed:

Master



SSH



Agent-01



jenkins user

SSH authentication was working correctly.

11. Configure Agent-01 in Jenkins

In Jenkins:

Manage Jenkins



Nodes



New Node

The node was created as:

Name: jenkins-agent-01

Type: Permanent Agent

The following configuration was used:

Remote root directory:

/home/jenkins

Executors:

1

Label:

agent-01

The node was configured to run only jobs matching the agent-01 label.

12. Configure SSH Launcher for Agent-01

The launch method was:

Launch agents via SSH

Configuration:

Host: 172.31.6.231

Port: 22

Credentials: ssh

The Jenkins SSH credential contained:

Kind:

SSH Username with private key

Username:

jenkins

The private key configured in Jenkins was the Master's:

jenkins_agent_key

Observation

Jenkins successfully authenticated to Agent-01.

The Jenkins log showed:

Authentication successful.

It then transferred:

remoting.jar

to the agent and started the Jenkins agent process.

The final message was:

Agent successfully connected and online

Therefore, Agent-01 was successfully connected to the Jenkins Master.

13. Test SSH Connection to Agent-02

The private IP of Agent-02 was:

172.31.28.245

From the Jenkins Master, the following command was used:

```
ssh -o IdentitiesOnly=yes -i ~/.ssh/jenkins_agent_key jenkins@172.31.28.245
```

The connection successfully opened:

```
[jenkins@ip-172-31-28-245 ~]$
```

Observation

This confirmed that:

Jenkins Master

↓

SSH authentication

↓

Agent-02

↓

jenkins user

was working correctly.

14. Configure Agent-02 in Jenkins

A second permanent node was created:

Name:

jenkins-agent-02

Configuration:

Remote root directory:

/home/jenkins

Executors:

1

Label:

agent-02

The usage option was configured as:

Only build jobs with label expressions matching this node

The SSH launcher was configured:

Host: 172.31.28.245

Port: 22

Credentials: ssh

The same SSH credential used for Agent-01 was used because both agents have the corresponding public key in their `authorized_keys`.

15. Jenkins Agent Communication Process

The complete connection process works as follows:

Step 1

Jenkins Master receives a job

↓

Step 2

Jenkins identifies an available agent

↓

Step 3

Jenkins connects to the agent using SSH

↓

Step 4

SSH authenticates the jenkins user



Step 5

Jenkins transfers remoting.jar



Step 6

Agent starts Java process



Step 7

Jenkins Remoting establishes communication



Step 8

Agent becomes ONLINE



Step 9

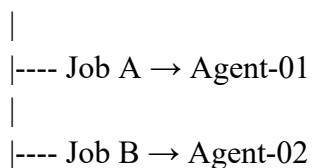
Jenkins can execute jobs on that agent

16. Why Jenkins Agents Are Used

The Jenkins Master is mainly responsible for **orchestration and management**, while agents perform the actual workload.

For example:

Jenkins Master



This allows different jobs to execute on different machines.

Labels can also control where a job runs:

Job 1

Label: agent-01



Agent-01

Job 2

Label: agent-02



Agent-02

Observations

1. Two Amazon Linux 2023 EC2 instances were configured as Jenkins agents.
2. A dedicated jenkins Linux user was created on both agents.
3. Java 21 was installed on the agents.
4. SSH key-based authentication was configured between the Jenkins Master and agents.
5. The private SSH key remained on the Jenkins Master.
6. The corresponding public key was configured in each agent's `authorized_keys`.
7. SSH connectivity from the Master to both agents was verified.
8. Agent-01 successfully connected to Jenkins and became **ONLINE**.
9. Agent-02 SSH connectivity was successfully verified and the Jenkins node was configured.
10. Jenkins uses `remoting.jar` and Java to establish communication with the agents.
11. The warning regarding JENKINS-42959 did not prevent Agent-01 from connecting successfully.

```
asifs@Asif_Shaik MINGW64 ~/downloads (master)
```

```
$ ssh -i jenkins.pem ec2-user@98.86.118.197
```

```
The authenticity of host '98.86.118.197 (98.86.118.197)' can't be established.
```

```
ED25519 key fingerprint is SHA256:Q8ZwdmmdrfONKjurnPy1RTCBdCdTcvgYv11x1svBCUM.
```

This key is not known by any other names

```
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

```
Warning: Permanently added '98.86.118.197' (ED25519) to the list of known hosts.
```

Amazon Linux 2023

<https://aws.amazon.com/linux/amazon-linux-2023>

```
[ec2-user@ip-172-31-6-126 ~]$ cat /etc/os-release
```

```
NAME="Amazon Linux"
```

```
VERSION="2023"
```

ID="amzn"

```
ID_LIKE="fedora"
```

VERSION_ID="2023"

```
PLATFORM_ID="platform:al2023"
```

```
PRETTY_NAME="Amazon Linux 2023.12.20260928"
ANSI_COLOR="0;32"
```

```
ANSI_COLOR="0;33"  
GRE_NAME="gre:3.3"
```

```
CPE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
```

```
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
```

```
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
```

```
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazonlinux-2-bugs/issues"
```

```
BUG_REPORT_URL= https://github.com/amazonlinux/amazon-linux-2023
VENDOR_NAME="AWS"
```

```
VENDOR_NAME="AWS"
VENDOR_URL="https
```

VENDOR_URL= https://aws.amazon.com/
SUPPORT_END="2029-06-30"

[ec2-user@ip-172-31-6-12]

```
lec2-user@ip-172-31-6-126 ~]$ java -version
-bash: java: command not found
```

```
-bash: java: command not found
[ec2-user@ip-172-31-6-126 ~]$
```

```
lec2-user@ip-172-31-6-126 ~]$ sudo dnf install java-21-amazon-corretto -y
Amazon Linux 2023 Kernel livepatch repository 693 kB/s | 82 kB 00:00
```

Dependencies resolved.

Package	Arch	Version	Repository	Size
Installing:				
java-21-amazon-corretto	x86_64	1:21.0.12+9-1.amzn2023.1	amazonlinux	217 k
Installing dependencies:				
alsa-lib	x86_64	1.2.7.2-1.amzn2023.0.4	amazonlinux	503 k
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux	717 k
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.3 M
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	467 k
dejavu-serif-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.0 M
fontconfig	x86_64	2.13.94-2.amzn2023.0.2	amazonlinux	273 k
fonts-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux	9.5 k
freetype	x86_64	2.13.2-5.amzn2023.0.2	amazonlinux	422 k
giflib	x86_64	5.2.1-9.amzn2023.0.4	amazonlinux	49 k
google-noto-fonts-common	noarch	20240401-1.amzn2023.0.2	amazonlinux	17 k
google-noto-sans-vf-fonts	noarch	20240401-1.amzn2023.0.2	amazonlinux	593 k
graphite2	x86_64	1.3.14-7.amzn2023.0.3	amazonlinux	97 k

```
[ec2-user@ip-172-31-6-126 ~]$ getent passwd jenkins
```

```
jenkins:x:993:993:Jenkins Automation Server:/var/lib/jenkins:/bin/false
```

```
[ec2-user@ip-172-31-6-126 ~]$ sudo usermod -s /bin/bash jenkins
```

```
[ec2-user@ip-172-31-6-126 ~]$ getent passwd jenkins
```

```
jenkins:x:993:993:Jenkins Automation Server:/var/lib/jenkins:/bin/bash
```

```
[ec2-user@ip-172-31-6-126 ~]$ sudo su - jenkins
```

```

Last login: Tue Sep 29 10:44:37 UTC 2026 on pts/3

```

```
[jenkins@ip-172-31-6-126 ~]$ /var/lib/jenkins
```

```
-bash: /var/lib/jenkins: Is a directory
```

```
[jenkins@ip-172-31-6-126 ~]$ client_loop: send disconnect: Connection reset by peer
```

```
asifs@Asif_Shaik MINGW64 ~/downloads (master)
```

We trust you have received the usual lecture from the local System Administrator. It usually boils down to these three things:

Last login: Tue Sep 29 13:43:59 2026

```
Sep 29 10:33:18 ip-172-31-6-231.ec2.internal systemd[1]: Starting sshd.service >
Sep 29 10:33:18 ip-172-31-6-231.ec2.internal sshd[1901]: Server listening on 0.>
Sep 29 10:33:18 ip-172-31-6-231.ec2.internal sshd[1901]: Server listening on ::
```



```

Complete!
[ec2-user@ip-172-31-28-245 ~]$ sudo systemctl enable --now sshd
[ec2-user@ip-172-31-28-245 ~]$ sudo mkdir -p /home/jenkins/.ssh
[ec2-user@ip-172-31-28-245 ~]$ sudo chown jenkins:jenkins /home/jenkins/.ssh
[ec2-user@ip-172-31-28-245 ~]$ sudo chmod 700 /home/jenkins/.ssh
[ec2-user@ip-172-31-28-245 ~]$ sudo vi /home/jenkins/.ssh/authorized_keys
[ec2-user@ip-172-31-28-245 ~]$ sudo chown jenkins:jenkins /home/jenkins/.ssh/authorized_keys
[ec2-user@ip-172-31-28-245 ~]$ sudo chmod 600 /home/jenkins/.ssh/authorized_keys
[ec2-user@ip-172-31-28-245 ~]$ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-28-245 ~]$ client_loop: send disconnect: Connection reset by peer

```

← → ↻ Not secure 98.86.118.197:8080/computer/ ☆ 📄 🌐 ⋮

 **Jenkins** / Nodes 🔍 ⚙️ 👤

Nodes

+ New Node Configure Monitors ↻

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	17.54 GiB	0 B	451.84 MiB	0ms
	jenkins-agent-01	Linux (amd64)	In sync	5.98 GiB	0 B	456.43 MiB	25ms
	jenkins-agent-02	Linux (amd64)	In sync	5.98 GiB	0 B	456.42 MiB	37ms
Data obtained		2 min 46 sec	2 min 46 sec	2 min 46 sec	2 min 46 sec	2 min 46 sec	2 min 46 sec

Icons: S M L Legend

REST API Jenkins 2.568.3

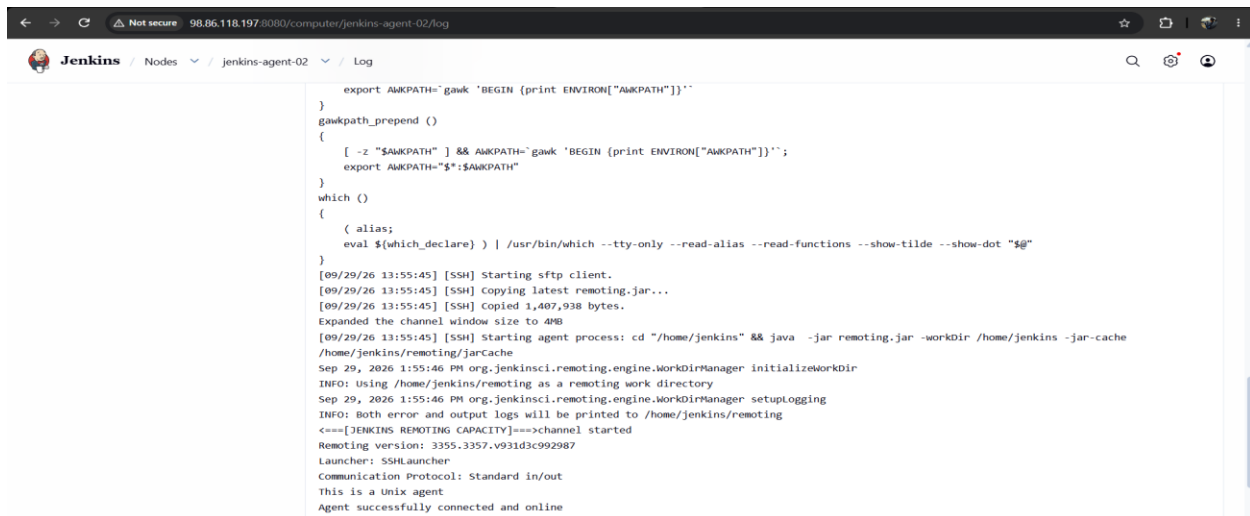
← → ↻ Not secure 98.86.118.197:8080/computer/jenkins-agent-01/log 🔍 ⚙️ 👤

 **Jenkins** / Nodes / jenkins-agent-01 / Log

```

}
gawkpath_prepend ()
{
    [ -z "$AWKPATH" ] && AWKPATH="gawk `BEGIN {print ENVIRON["AWKPATH"]}`";
    export AWKPATH="$*: $AWKPATH"
}
which ()
{
    ( alias;
    eval $(which_declare) ) | /usr/bin/which --tty-only --read-alias --read-functions --show-tilde --show-dot "$@"
}
[09/29/26 13:34:56] [SSH] Starting sftp client.
[09/29/26 13:34:56] [SSH] Copying latest remoting.jar...
Source agent hash is B7A48EA7B03DA66668806FBC925CD6F8. Installed agent hash is B7A48EA7B03DA66668806FBC925CD6F8
Verified agent jar. No update is necessary.
Expanded the channel window size to 4MB
[09/29/26 13:34:56] [SSH] Starting agent process: cd "/home/jenkins" && java -jar remoting.jar -workDir /home/jenkins -jar-cache /home/jenkins/remoting/jar-cache
Sep 29, 2026 1:34:56 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/remoting as a remoting work directory
Sep 29, 2026 1:34:56 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/remoting
====[JENKINS REMOTING CAPACITY]====channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online

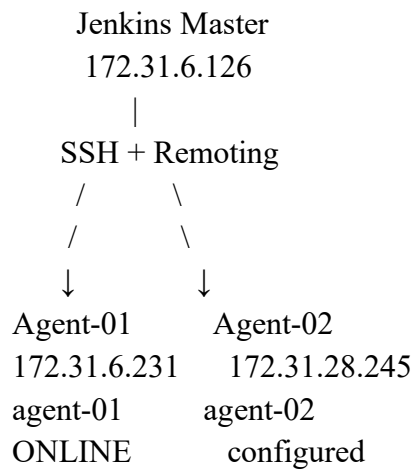
```



```
export AWKPATH="gawk 'BEGIN {print ENVIRON["AWKPATH"]}''"
}
gawkpath_prepend ()
{
[ -z "$AWKPATH" ] && AWKPATH="gawk 'BEGIN {print ENVIRON["AWKPATH"]}''";
export AWKPATH="$*: $AWKPATH"
}
which ()
{
( alias;
eval $(which_declare) ) | /usr/bin/which --tty-only --read-alias --read-functions --show-tilde --show-dot "$@"
}
[09/29/26 13:55:45] [SSH] Starting sftp client.
[09/29/26 13:55:45] [SSH] Copying latest remoting.jar...
[09/29/26 13:55:45] [SSH] Copied 1,407,938 bytes.
Expanded the channel window size to 4MB
[09/29/26 13:55:45] [SSH] Starting agent process: cd "/home/jenkins" && java -jar remoting.jar -workDir /home/jenkins -jar-cache
/home/jenkins/remoting/jarCache
Sep 29, 2026 1:55:46 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/remoting as a remoting work directory
Sep 29, 2026 1:55:46 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/remoting
<==[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHTLSLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

Result

Two Jenkins agent machines were successfully prepared and connected/configured with the Jenkins Master using SSH-based communication.



Conclusion

The Jenkins Master and agent architecture was successfully implemented. The agents use a dedicated jenkins Linux account and communicate with the Master through SSH and Jenkins Remoting. Java 21 provides the runtime required by the Jenkins agent process, while SSH public-key authentication provides secure Master-to-agent access.

This configuration allows Jenkins to distribute and execute jobs on separate machines instead of performing all job execution directly on the Master.

Task 2 — Configure GitHub Webhook to Jenkins Job

Objective

To configure a GitHub webhook so that whenever a code change is pushed to the GitHub repository, Jenkins automatically receives the webhook notification and triggers the configured Jenkins job.

Steps Used

1. Created a GitHub Repository

Created a new public GitHub repository:

jenkins-webhook-practice

Repository URL:

<https://github.com/asifshaik6558/jenkins-webhook-practice.git>

The repository was initialized with a README file.

2. Installed Git on Jenkins Master

Connected to the Jenkins master server and installed Git:

```
sudo dnf install git -y
```

Verified the installation:

```
git --version
```

Also verified that the Jenkins user could execute Git:

```
sudo -u jenkins git --version
```

3. Installed Git on Jenkins Agent-02

The Jenkins job was configured to execute on jenkins-agent-02. Therefore, Git was also installed on Agent-02:

```
sudo dnf install git -y
```

Verified:

```
git --version
```

4. Configured Git in Jenkins

Navigated to:

Jenkins → **Manage Jenkins** → **Tools**

Configured the Git installation:

Name: Git

Path to Git executable: /usr/bin/git

5. Configured the Jenkins Job

Created/configured the Freestyle job:

github-webhook-test

Under **Source Code Management** → **Git**, configured:

Repository URL:

<https://github.com/asifshaik6558/jenkins-webhook-practice.git>

Credentials:

None

Branch:

*/main

Since the repository was public, no GitHub credentials were required for cloning.

6. Configured the Jenkins Build Trigger

Under **Build Triggers**, enabled:

GitHub hook trigger for GITScm polling

Poll SCM was not enabled because it is covered separately in the next task.

7. Allowed External Webhook Traffic

Updated the Jenkins master's EC2 Security Group to allow:

Type: Custom TCP

Port: 8080

Source: 0.0.0.0/0

This allowed GitHub to reach the Jenkins webhook endpoint.

8. Configured GitHub Webhook

Navigated to:

GitHub Repository → Settings → Webhooks → Add webhook

Configured the payload URL:

<http://<Jenkins-Master-Public-IP>:8080/github-webhook/>

Configured:

Content type: application/json

Selected:

Just the push event

Enabled:

Active

After saving the webhook, GitHub displayed a **green check mark**, confirming successful delivery to Jenkins.

9. Tested the Webhook

Modified the repository README and committed the change to the main branch.

The GitHub push automatically triggered the Jenkins job.

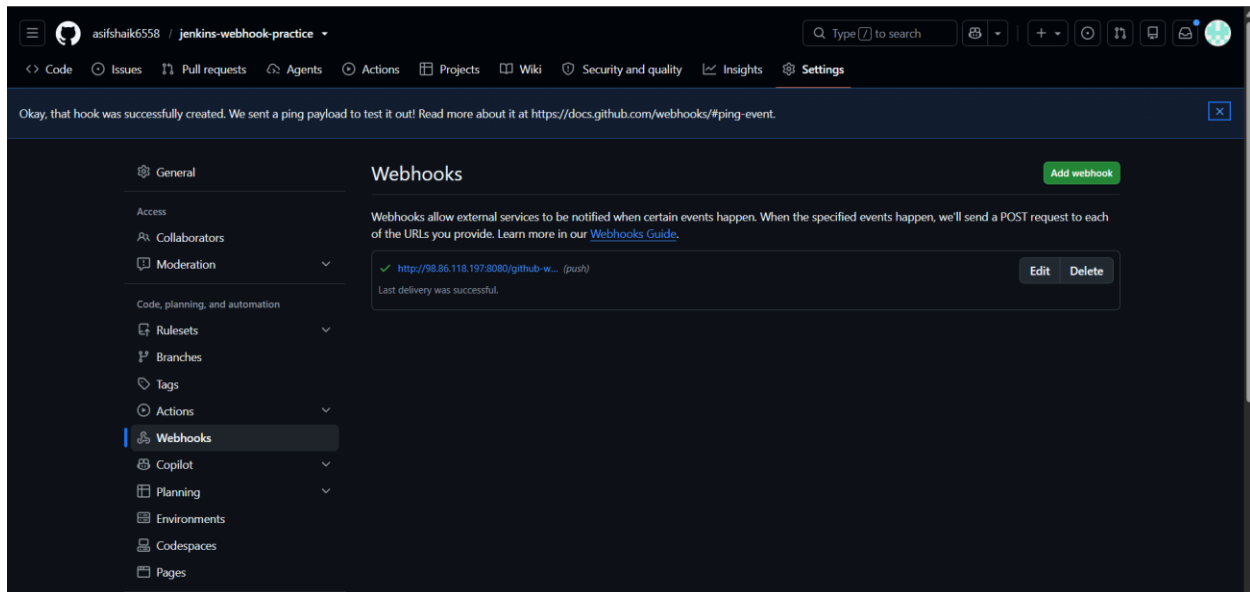
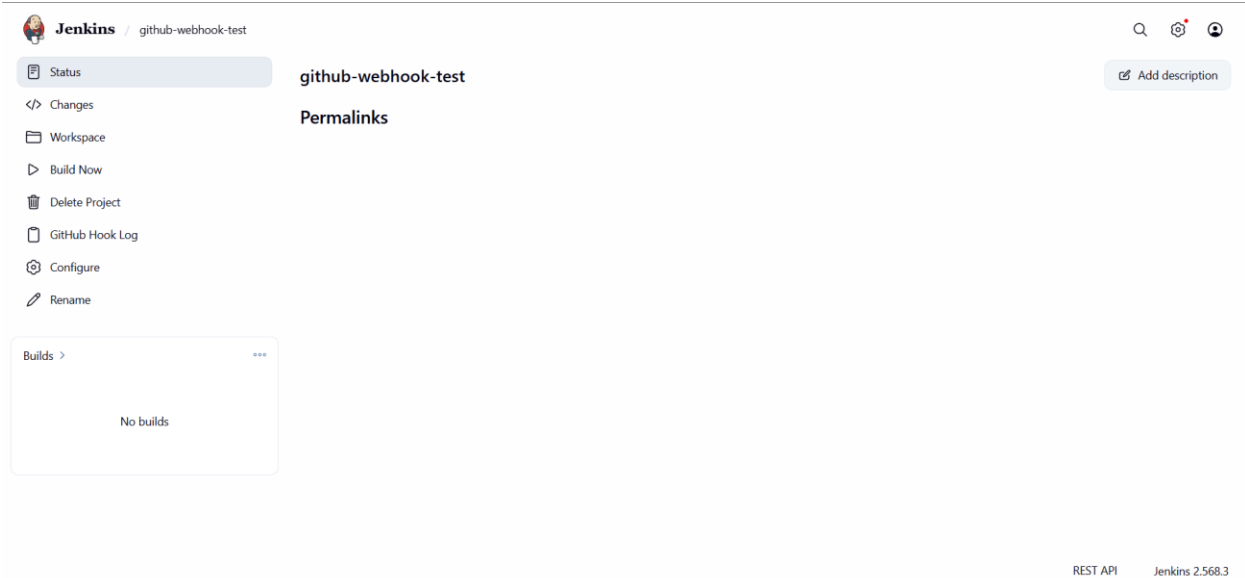
The Jenkins console showed:

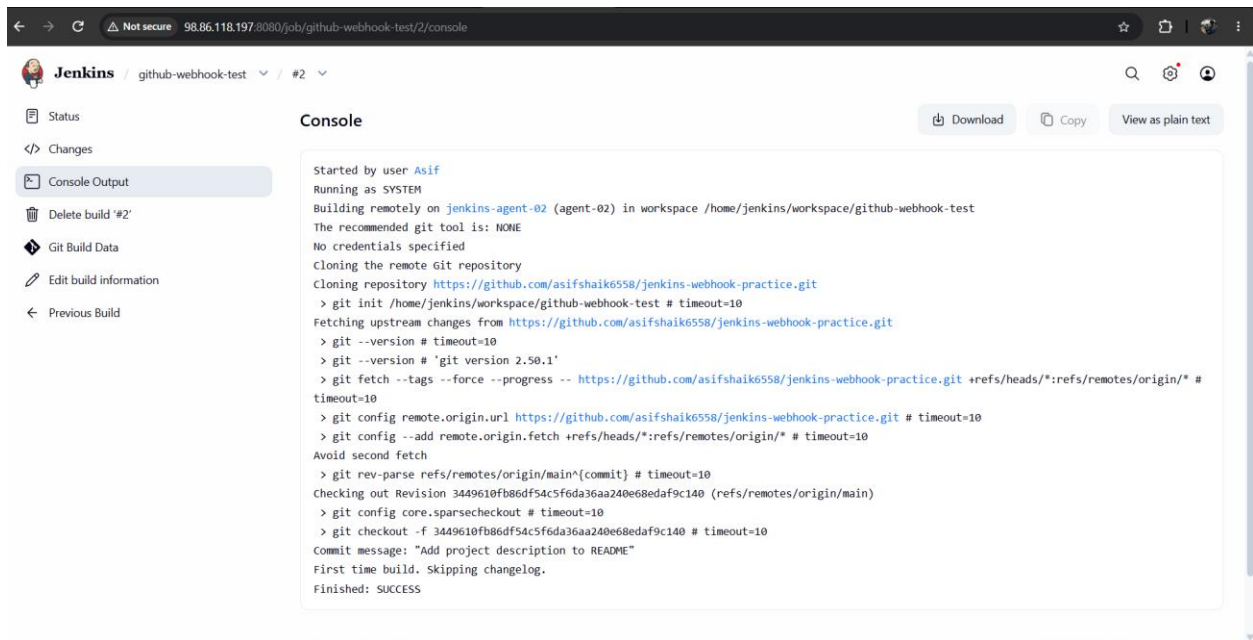
Started by GitHub push by asifshaik6558

The job was executed remotely on:

jenkins-agent-02

The job then cloned the GitHub repository using Git installed on Agent-02.





Observation

The first webhook-triggered build reached Agent-02 successfully but failed because Git was not installed on Agent-02:

Cannot run program "git"

Git was then installed on Agent-02 and verified successfully.

After installing Git on the agent, the Jenkins job was able to perform the Git checkout.

Result

The GitHub webhook was successfully configured and connected to the Jenkins job.

The workflow achieved was:

Developer Push



GitHub Repository



GitHub Webhook



Jenkins Master
↓
Jenkins Job
↓
Jenkins Agent-02
↓
Git Clone

Conclusion

The GitHub webhook successfully automated the triggering of the Jenkins job whenever a push was made to the GitHub repository. The task also demonstrated that required tools such as Git must be installed on the Jenkins agent where the job actually executes.

Task 3 — Configure Poll SCM and Build Periodically.

Objective

To configure Jenkins build triggers using **Poll SCM** and **Build Periodically**, and understand the difference between them.

Steps Used

1. Opened the Jenkins Job

Navigated to:

Jenkins Dashboard
→ github-webhook-test
→ Configure

2. Configured Poll SCM

Under **Build Triggers**, selected:

☒ Poll SCM

Configured the schedule:

H/5 * * * *

This makes Jenkins check the configured Git repository approximately every 5 minutes.

Poll SCM triggers a build **only when Jenkins detects changes in the source repository**.

3. Configured Build Periodically

Under **Build Triggers**, selected:

☒ Build periodically

Configured:

H/5 * * * *

This makes Jenkins execute the job approximately every 5 minutes **regardless of whether there are source-code changes**.

4. Saved the Configuration

Clicked **Save** after configuring both triggers.

Difference Between Poll SCM and Build Periodically

Feature	Poll SCM	Build Periodically
Checks source repository	Yes	No
Detects source-code changes	Yes	No
Builds only when changes exist	Yes	No
Builds according to schedule	Yes	Yes
Example	Check every 5 minutes and build if code changed	Build every 5 minutes regardless of changes

Simple Example

Poll SCM:

Every 5 minutes



Check GitHub



Code changed?



Yes



No



Build



No Build

Build Periodically:

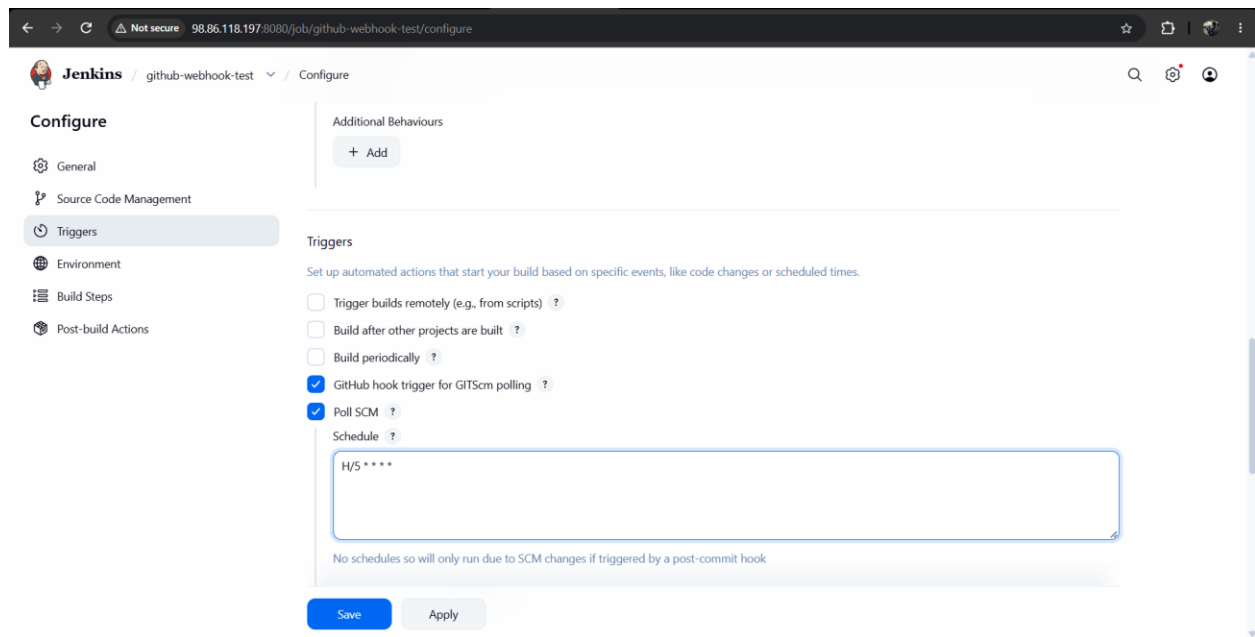
Every 5 minutes

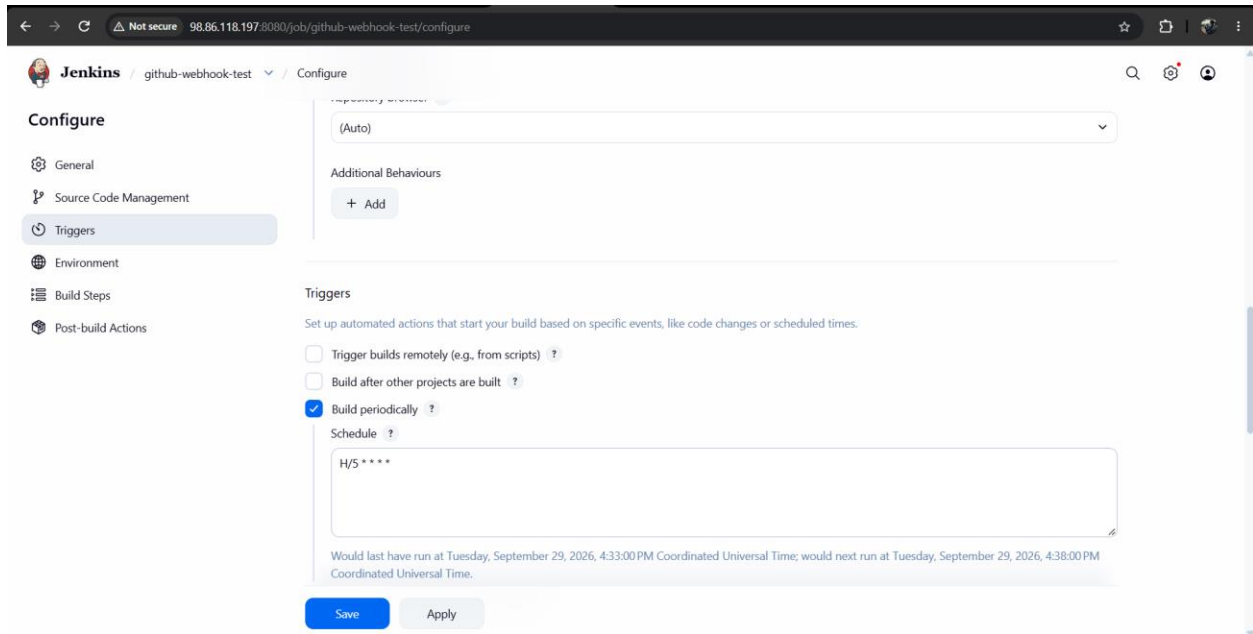


Build



No need to check whether code changed





Observation

Both triggers were successfully configured in the github-webhook-test Jenkins job using the schedule:

H/5 * * * *

The **Poll SCM** trigger is change-based, whereas **Build Periodically** is time-based.

The GitHub webhook trigger configured in Task 2 remains separate and can trigger the job immediately when GitHub sends a push event.

Result

Poll SCM and Build Periodically were successfully configured for the Jenkins job.

Conclusion

This task demonstrated three different ways to trigger a Jenkins job:

GitHub Webhook → Trigger on push

Poll SCM → Check for changes → Build if changed

Build Periodically → Build according to schedule

The key difference is that **Poll SCM checks for source-code changes before deciding to build, while Build Periodically runs the build based only on the configured schedule.**

Task 4 — Create a Bash script to back up the Jenkins server.

Objective

To create a Bash script that takes a backup of the Jenkins server's **Jenkins Home directory** (/var/lib/jenkins) and stores the backup as a compressed, timestamped .tar.gz file under /backup.

Step 1 — Connect to the Jenkins Master

Connected to the newly created Jenkins master using SSH:

```
ssh -i jenkins.pem ec2-user@18.234.217.59
```

Observation

Successfully connected to the Jenkins master:

```
[ec2-user@ip-172-31-25-214 ~]$
```

The new Jenkins master was accessible through SSH.

Step 2 — Identify the Jenkins Home Directory

The Jenkins installation stores its main configuration and data under:

```
/var/lib/jenkins
```

This directory contains important Jenkins data such as:

```
/var/lib/jenkins/  
├── jobs/  
├── plugins/  
├── users/  
├── nodes/  
├── secrets/  
└── config.xml
```

- credentials.xml
- other Jenkins configuration files

Observation

/var/lib/jenkins was selected as the source directory for the backup because it contains the Jenkins server configuration and data.

Step 3 — Create the Backup Script

Created a Bash script:

```
vi ~/jenkins_backup.sh
```

The following script was added:

```
#!/bin/bash
```

```
BACKUP_DIR=/backup
```

```
TIMESTAMP=$(date +%F_%H-%M-%S)
```

```
mkdir -p "$BACKUP_DIR"
```

```
tar -czf "$BACKUP_DIR/jenkins_backup_$TIMESTAMP.tar.gz" -C /var/lib jenkins
```

```
echo "Jenkins backup completed successfully."
```

Saved the file using:

```
Esc
```

```
:wq
```

```
Enter
```

Observation

The Bash script was successfully created in the ec2-user home directory.

Step 4 — Make the Script Executable

Changed the script permissions:

```
chmod +x ~/jenkins_backup.sh
```

Verified the permissions:

```
ls -l ~/jenkins_backup.sh
```

Observation

The script received execute permission.

Example:

```
-rwxr-xr-x
```

The x permission confirms that the script can be executed.

Step 5 — Execute the Backup Script

Executed the script using elevated privileges:

```
sudo ~/jenkins_backup.sh
```

Observation

The script displayed:

Jenkins backup completed successfully.

This confirmed that the backup command completed successfully.

Step 6 — Backup Directory Creation

The script automatically created the backup directory:

```
/backup
```

This was done using:

```
mkdir -p "$BACKUP_DIR"
```

Observation

The /backup directory was created and used as the destination for the Jenkins backup.

Step 7 — Create Compressed Jenkins Backup

The script used the following command:

```
tar -czf "$BACKUP_DIR/jenkins_backup_${TIMESTAMP}.tar.gz" -C /var/lib jenkins
```

The command performs the following:

tar

↓

Creates an archive

↓

-z

↓

Compresses using gzip

↓

-f

↓

Creates the specified backup file

The backup filename automatically included the date and time.

The generated backup was:

```
jenkins_backup_2026-09-30_06-53-21.tar.gz
```

Observation

The Jenkins Home directory was successfully compressed into a single .tar.gz archive.

Step 8 — Verify the Backup File

Checked the contents of the /backup directory:

```
sudo ls -lh /backup/
```

Output:

```
total 145M
-rw-r--r--. 1 root root 145M Sep 30 06:53 jenkins_backup_2026-09-30_06-53-21.tar.gz
```

Observation

The backup file was successfully created.

Backup size: 145 MB

Backup format: .tar.gz

Backup location:

/backup/jenkins_backup_2026-09-30_06-53-21.tar.gz

Step 9 — Verify Backup Contents

The backup archive was inspected using:

```
sudo tar -tzf /backup/jenkins_backup_2026-09-30_06-53-21.tar.gz | head -20
```

The output included:

```
jenkins/
jenkins/.java/
jenkins/.java/fonts/
jenkins/secret.key
jenkins/secret.key.not-so-secret
jenkins/plugins/
jenkins/plugins/commons-lang3-api.jpi
jenkins/plugins/commons-lang3-api/
...
```

```
[ec2-user@ip-172-31-25-214 ~]$ vi ~/jenkins_backup.sh
[ec2-user@ip-172-31-25-214 ~]$ chmod +x ~/jenkins_backup.sh
[ec2-user@ip-172-31-25-214 ~]$ ls -l ~/jenkins_backup.sh
-rwxr-xr-x. 1 ec2-user ec2-user 211 Sep 30 06:52 /home/ec2-user/jenkins_backup.sh
[ec2-user@ip-172-31-25-214 ~]$ sudo ~/jenkins_backup.sh
Jenkins backup completed successfully.
[ec2-user@ip-172-31-25-214 ~]$ sudo ls -lh /backup/
total 145M
-rw-r--r--. 1 root root 145M Sep 30 06:53 jenkins_backup_2026-09-30_06-53-21.tar.gz
[ec2-user@ip-172-31-25-214 ~]$ sudo tar -tzf /backup/jenkins_backup_2026-09-30_06-53-21.tar.gz | head -20
jenkins/
jenkins/.java/
jenkins/.java/fonts/
jenkins/.java/fonts/21.0.12.1/
jenkins/.java/fonts/21.0.12.1/fcinfo-1-ip-172-31-25-214.ec2.internal-amzn-2023-en-.properties
jenkins/secret.key
jenkins/secret.key.not-so-secret
jenkins/plugins/
jenkins/plugins/commons-lang3-api.jpi
jenkins/plugins/commons-lang3-api/
jenkins/plugins/commons-lang3-api/META-INF/
jenkins/plugins/commons-lang3-api/META-INF/MANIFEST.MF
jenkins/plugins/commons-lang3-api/META-INF/maven/
jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/
jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/commons-lang3-api/
jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/commons-lang3-api/pom.xml
jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/commons-lang3-api/pom.properties
jenkins/plugins/commons-lang3-api/WEB-INF/
jenkins/plugins/commons-lang3-api/WEB-INF/lib/
jenkins/plugins/commons-lang3-api/WEB-INF/lib/commons-lang3-3.20.0.jar
```

```
#!/bin/bash
```

```
BACKUP_DIR=/backup
```

```

TIMESTAMP=$(date +%F_%H-%M-%S)

```

```
mkdir -p "$BACKUP_DIR"
```

```
tar -czf "$BACKUP_DIR/jenkins_backup_${TIMESTAMP}.tar.gz" -C /var/lib jenkins
```

```
echo "Jenkins backup completed successfully."
```


2

2

2

22

2

Observation

The archive contained the Jenkins directory and important Jenkins files and plugins.

This confirmed that the backup was not empty and that the Jenkins Home directory had been successfully archived.

Backup Process

The complete process performed was:

Jenkins Master



/var/lib/jenkins



jenkins_backup.sh



tar + gzip



Timestamped backup



/backup/



jenkins_backup_2026-09-30_06-53-21.tar.gz



Verify file



Verify contents

Result

The Jenkins server backup was successfully created using a Bash script.

The final backup file was:

/backup/jenkins_backup_2026-09-30_06-53-21.tar.gz

Size: 145 MB

The archive contents were also verified successfully.

Conclusion

Task 4 — Bash Script Backup of Jenkins Server was successfully completed.

A reusable Bash script was created to compress the Jenkins Home directory /var/lib/jenkins into a timestamped .tar.gz backup. The generated backup was verified for both file existence and contents, confirming that the Jenkins data was successfully backed up.

Task 5 — Take a backup of Jenkins using the ThinBackup plugin

Objective

To install and configure the **ThinBackup plugin** in Jenkins and create a backup of the Jenkins configuration and related data.

Steps Used

Step 1: Install ThinBackup Plugin

1. Open Jenkins dashboard.
2. Go to **Manage Jenkins**.
3. Select **Plugins**.
4. Open the **Available plugins** section.
5. Search for **ThinBackup**.
6. Select the **ThinBackup** plugin.
7. Click **Install**.
8. Wait until the installation shows **Success**.

Observation:

ThinBackup and its required dependencies were installed successfully.

Step 2: Open ThinBackup Configuration

1. Go to **Manage Jenkins**.
2. Open **System**.
3. Scroll down to the **ThinBackup Configuration** section.

Observation:

The ThinBackup configuration was available under the Jenkins global system configuration.

Step 3: Configure Backup Directory

In **Backup directory**, entered:

/backup/thinbackup

Jenkins initially displayed:

The directory does not exist, but will be created before the first run.

This was not treated as an error because ThinBackup can create the directory during the backup.

Clicked **Save**.

Step 4: Check Backup Directory Permissions

From the Jenkins master server, checked the /backup directory:

```
sudo ls -ld /backup
```

Observation:

```
drwxr-xr-x. 2 root root ...
```

The directory was owned by root, while Jenkins runs using the jenkins user.

Step 5: Change Ownership of Backup Directory

Changed the ownership of /backup to the Jenkins user:

```
sudo chown jenkins:jenkins /backup
```

Verified the ownership:

```
sudo ls -ld /backup
```

Observation:

```
drwxr-xr-x. 2 jenkins jenkins ...
```

The /backup directory was now accessible to the Jenkins service user.

Step 6: Run ThinBackup Manually

1. Go to **Manage Jenkins** → **ThinBackup**.
2. Clicked **Backup now**.
3. Waited for the backup operation to complete.

Step 7: Verify ThinBackup

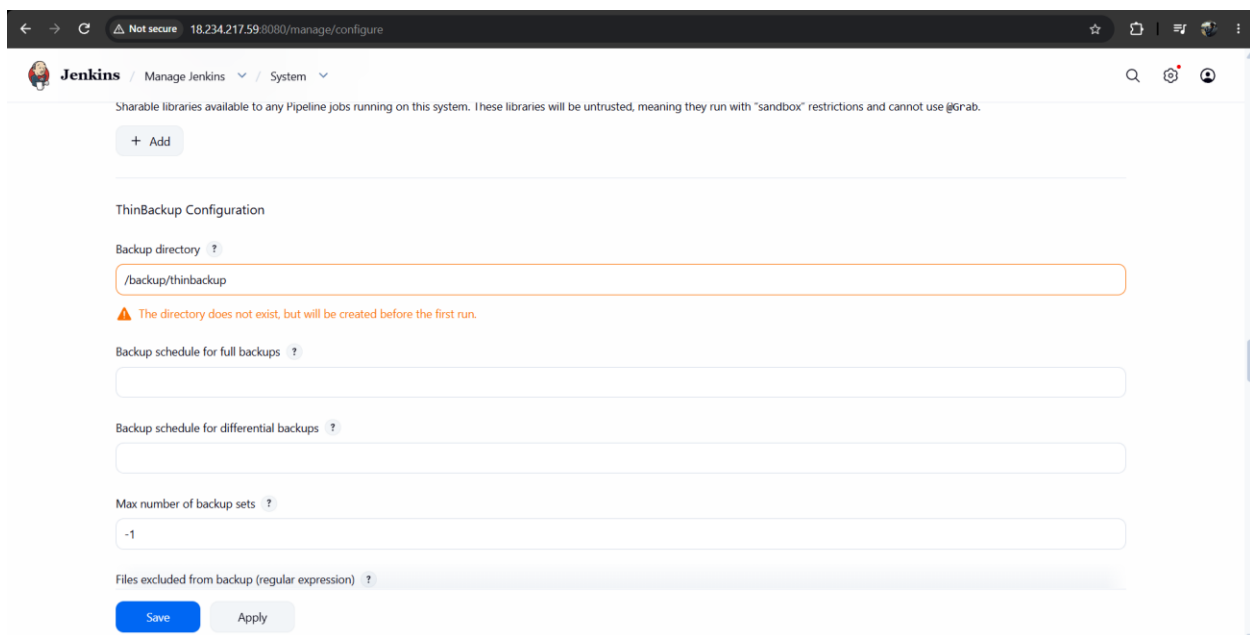
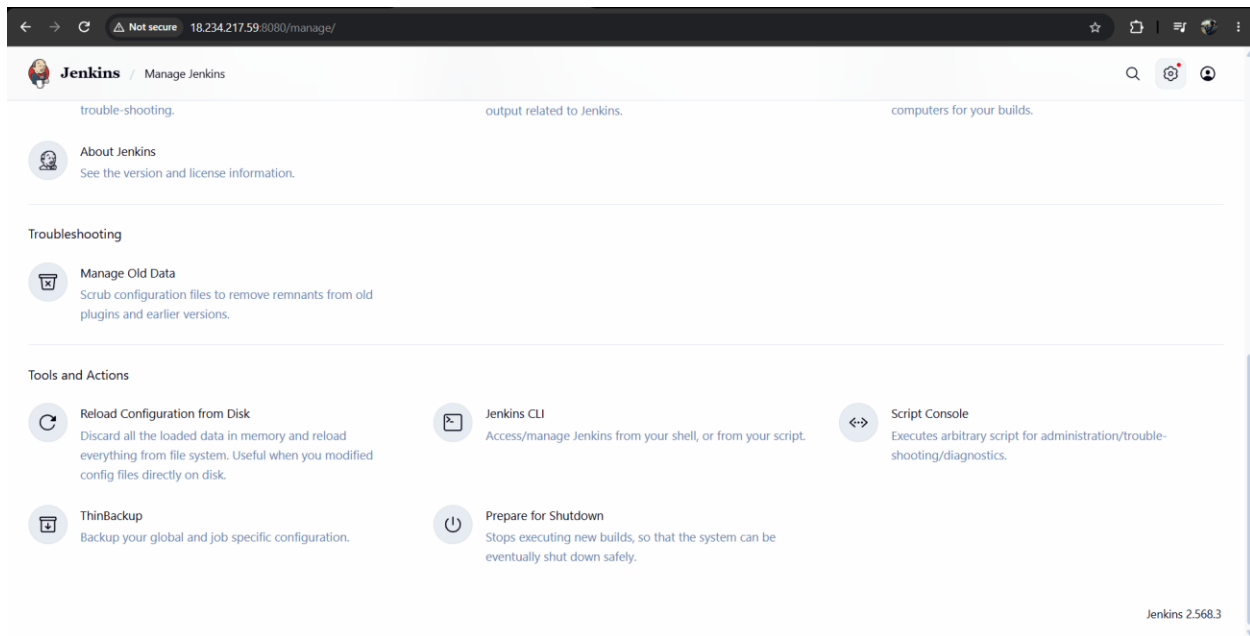
On the Jenkins master, executed:

```
sudo ls -lh /backup/thinbackup
```

Observation:

```
total 16K
drwxr-xr-x. 4 jenkins jenkins 16K Sep 30 07:15 FULL-2026-09-30_07-15
```

The FULL-2026-09-30_07-15 directory was created successfully.



```
[ec2-user@ip-172-31-25-214 ~]$ sudo chown jenkins:jenkins /backup
[ec2-user@ip-172-31-25-214 ~]$ sudo ls -ld /backup
drwxr-xr-x. 2 jenkins jenkins 55 Sep 30 06:53 /backup
[ec2-user@ip-172-31-25-214 ~]$ sudo ls -lh /backup/thinbackup
total 16K
drwxr-xr-x. 4 jenkins jenkins 16K Sep 30 07:15 FULL-2026-09-30_07-15
```

Result

The **ThinBackup** plugin was successfully installed, configured, and tested. A full Jenkins backup was successfully created under:

/backup/thinbackup/FULL-2026-09-30_07-15

Conclusion

ThinBackup provides a Jenkins-native method for backing up Jenkins configuration and related data. The backup directory was configured, the required permissions were corrected for the jenkins user, and a **full backup was successfully generated and verified**.

Task 6 — Set up a new Jenkins server and restore the backup taken in task 4.

Objective

To launch a **new Jenkins server**, install Jenkins, transfer the backup created in **Task 4**, restore the backup, and verify that the previous Jenkins configuration is available on the new server.

Steps Used

Step 1: Launch a New EC2 Instance

A separate EC2 instance was launched for the Jenkins restore task.

Configuration used:

- **Instance name:** jenkins-restore
- **AMI:** Amazon Linux 2023
- **Instance type:** t3.micro
- **Key pair:** jenkins.pem
- **Security Group:**
 - SSH — Port 22
 - Jenkins — Port 8080

Observation:

A new EC2 instance was successfully created specifically for restoring the Jenkins backup.

Step 2: Connect to the New Server

Connected to the new EC2 instance using SSH:

```
ssh -i jenkins.pem ec2-user@13.220.179.106
```

Observation:

Successfully connected to the new Jenkins restore server.

Step 3: Install Java 21

Installed Amazon Corretto Java 21:

```
sudo dnf install java-21-amazon-corretto -y
```

Observation:

Java 21 was successfully installed on the new server.

Step 4: Install Jenkins

Added the official Jenkins repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

Imported the Jenkins repository key:

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

Installed Jenkins:

```
sudo dnf install jenkins -y
```

Observation:

Jenkins was successfully installed on the new EC2 instance.

Step 5: Start Jenkins

Started Jenkins:

```
sudo systemctl start jenkins
```

Verified the service:

```
sudo systemctl status jenkins
```

Observation:

Jenkins showed:

Active: active (running)

Step 6: Stop Jenkins Before Restoration

Stopped Jenkins before replacing its data with the backup:

```
sudo systemctl stop jenkins
```

Observation:

Jenkins was stopped successfully.

Step 7: Copy the Task 4 Backup to the New Server

The Task 4 backup was stored on the old Jenkins server:

```
/backup/jenkins_backup_2026-09-30_06-53-21.tar.gz
```

The backup was transferred from the old Jenkins server:

18.234.217.59

to the new Jenkins restore server:

13.220.179.106

using scp from Windows Git Bash:

```
scp -i jenkins.pem ec2-user@18.234.217.59:/backup/jenkins_backup_2026-09-30_06-53-21.tar.gz ec2-user@13.220.179.106:/home/ec2-user/
```

Observation:

The first transfer was incomplete, showing only **66 MB**, so that corrupted/incomplete file was deleted and the backup was transferred again.

The second transfer completed successfully.

Step 8: Verify the Backup

On the new Jenkins server, verified the transferred backup:

```
ls -lh /home/ec2-user/jenkins_backup_2026-09-30_06-53-21.tar.gz
```

Observation:

145M

The complete Task 4 backup was successfully transferred.

Step 9: Restore the Backup

The backup was extracted into /var/lib:

```
sudo tar -xzf /home/ec2-user/jenkins_backup_2026-09-30_06-53-21.tar.gz -C /var/lib
```

The backup contained the Jenkins directory, so the Jenkins data was restored to:

```
/var/lib/jenkins
```

Observation:

The Jenkins backup was successfully extracted.

Step 10: Fix Jenkins Ownership

Changed ownership of the restored Jenkins files:

```
sudo chown -R jenkins:jenkins /var/lib/jenkins
```

Observation:

The restored Jenkins files were assigned to the jenkins service user.

Step 11: Start Jenkins After Restoration

Started Jenkins:

```
sudo systemctl start jenkins
```

Verified that Jenkins was running.

Observation:

Active: active (running)

Step 12: Verify the Restoration

Opened the new Jenkins server:

<http://13.220.179.106:8080>

MINGW64~/c/Users/asifs/downloads

```
asifs@Asif-Shaik MINGW64 ~/downloads (master)
$ ssh -i jenkins.pem ec2-user@13.220.179.106
The authenticity of host '13.220.179.106 (13.220.179.106)' can't be established.
ED25519 key fingerprint is SHA256:/MgaSmbSH8vGtWrsuHwydSMAPKwcdFq8ZyQwZpFcIs.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.220.179.106' (ED25519) to the list of known hosts
```

```

  ____      _
 / ___|    / \   _ __| |__   __| |
| |  | |  / _ \ / _ \| '_ \|  _ \| | | | | | | | | |
| |  | | / ___ \| | | | |_) | |_) |
| |  | | / ___ \| | | | |_) | |_) |
|_|  |_|/_/___|_| |_|_|__|_|__|_|

Amazon Linux 2023

https://aws.amazon.com/linux/amazon-linux-2023
```

```
[ec2-user@ip-172-31-20-204 ~]$ sudo dnf install java-21-amazon-corretto -y
Amazon Linux 2023 Kernel Livepatch repository 733 kB/s | 82 kB 00:00
Dependencies resolved.
```

Package	Arch	Version	Repository	Size
Installing:				
java-21-amazon-corretto	x86_64	1:21.0.12+9-1.amzn2023.1	amazonlinux	217 k
Installing dependencies:				
alsa-lib	x86_64	1.2.7.2-1.amzn2023.0.4	amazonlinux	503 k
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux	717 k
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.3 M
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	467 k
dejavu-serif-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.0 M
fontconfig	x86_64	2.13.94-2.amzn2023.0.2	amazonlinux	273 k
fonts-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux	9.5 k
freetype	x86_64	2.13.2-5.amzn2023.0.2	amazonlinux	422 k
glib	x86_64	5.2.1-9.amzn2023.0.4	amazonlinux	49 k
google-noto-fonts-common	noarch	20240401-1.amzn2023.0.2	amazonlinux	17 k
google-noto-sans-vf-fonts	noarch	20240401-1.amzn2023.0.2	amazonlinux	593 k
graphite2	x86_64	1.3.14-7.amzn2023.0.3	amazonlinux	97 k
harfbuzz	x86_64	7.0.0-2.amzn2023.0.2	amazonlinux	873 k
java-21-amazon-corretto-headless	x86_64	1:21.0.12+9-1.amzn2023.1	amazonlinux	96 M
javapackages-filesystem	noarch	6.0.0-7.amzn2023.0.6	amazonlinux	12 k
langpacks-core-font-en	noarch	3.0-21.amzn2023.0.4	amazonlinux	10 k
libICE	x86_64	1.1.1-3.amzn2023.0.1	amazonlinux	76 k
libSM	x86_64	1.2.4-3.amzn2023.0.1	amazonlinux	45 k
libX11	x86_64	1.8.10-2.amzn2023.0.1	amazonlinux	659 k
libX11-common	noarch	1.8.10-2.amzn2023.0.1	amazonlinux	147 k
libXau	x86_64	1.0.11-6.amzn2023.0.1	amazonlinux	33 k
libXext	x86_64	1.3.6-1.amzn2023.0.1	amazonlinux	42 k
libXi	x86_64	1.8.2-1.amzn2023.0.1	amazonlinux	42 k
libXinerama	x86_64	1.1.5-6.amzn2023.0.1	amazonlinux	16 k
libXrandr	x86_64	1.5.4-3.amzn2023.0.1	amazonlinux	29 k
libXrender	x86_64	0.9.11-6.amzn2023.0.1	amazonlinux	29 k
libXt	x86_64	1.3.0-3.amzn2023.0.1	amazonlinux	183 k
libXtst	x86_64	1.2.5-1.amzn2023.0.1	amazonlinux	22 k
libbrotli	x86_64	1.1.0-1.amzn2023.0.1	amazonlinux	334 k

```
asifs@Asif-Shaik MINGW64 ~/downloads (master)
$ scp -i jenkins.pem ec2-user@13.220.179.106:/home/ec2-user/
jenkins_backup_2026-09-30_06-53-21.tar.gz ec2-user@13.220.179.106:/home/ec2-user/
100% 145MB 1.5MB/s 01:39
```

```
asifs@Asif-Shaik MINGW64 ~/downloads (master)
$ ssh -i jenkins.pem ec2-user@13.220.179.106

  ____      _
 / ___|    / \   _ __| |__   __| |
| |  | |  / _ \ / _ \| '_ \|  _ \| | | | | | | | | |
| |  | | / ___ \| | | | |_) | |_) |
| |  | | / ___ \| | | | |_) | |_) |
|_|  |_|/_/___|_| |_|_|__|_|__|_|

Amazon Linux 2023

https://aws.amazon.com/linux/amazon-linux-2023

Last login: Wed Sep 30 07:54:10 2026 from 103.160.27.229
[ec2-user@ip-172-31-20-204 ~]$ ls -lh /home/ec2-user/jenkins_backup_2026-09-30_06-53-21.tar.gz
-rw-r--r-- 1 ec2-user ec2-user 145M Sep 30 07:58 /home/ec2-user/jenkins_backup_2026-09-30_06-53-21.tar.gz
[ec2-user@ip-172-31-20-204 ~]$ sudo tar -xzf /home/ec2-user/jenkins_backup_2026-09-30_06-53-21.tar.gz -C /var/lib
[ec2-user@ip-172-31-20-204 ~]$ sudo chown -R jenkins:jenkins /var/lib/jenkins
[ec2-user@ip-172-31-20-204 ~]$ sudo systemctl start jenkins
[ec2-user@ip-172-31-20-204 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-09-30 08:00:25 UTC; 10s ago
     Main PID: 27464 (java)
       Tasks: 49 (limit: 1059)
        Memory: 303.8M
           CPU: 21.22s
      CGroup: /system.slice/jenkins.service
              └─27464 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.300+0000 [id=11] INFO hudson.slaves.SlaveComputer#tryReconnect: Attempting to reconnect Jenkins Agent 02
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.305+0000 [id=11] INFO hudson.slaves.SlaveComputer#tryReconnect: Attempting to reconnect jenkins-agent-01
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.312+0000 [id=33] INFO jenkins.InitReactorRunner$1#onAttained: System config loaded
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.313+0000 [id=33] INFO jenkins.InitReactorRunner$1#onAttained: System config adapted
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.316+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.351+0000 [id=33] INFO jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.439+0000 [id=32] WARNING hudson.model.User$AllUsers$CamAll: ignoring /var/lib/jenkins/users/admin_94986b4a6b9e1
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.490+0000 [id=30] INFO jenkins.InitReactorRunner$1#onAttained: Completed initialization
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal jenkins[27464]: 2026-09-30 08:00:25.558+0000 [id=24] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
Sep 30 08:00:25 ip-172-31-20-204.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
```

https://ow02041520

← → ↻ Not secure 18.234.217.59:8080

Jenkins

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

Built-in Node	(● offline)
Jenkins Agent 02	0/1
jenkins-agent-01	0/1

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

REST API Jenkins 2.568.3

← → ↻ Not secure 13.220.179.106:8080

Jenkins

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

Built-in Node	(● offline)
Jenkins Agent 02	(🔄 launching...)
jenkins-agent-01	(🔄 launching...)

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

REST API Jenkins 2.568.3

Observation:

The previous Jenkins configuration was visible on the new server, confirming that the backup had been successfully restored.

Result

A **separate new Jenkins server** was successfully launched and configured. The **145 MB Bash backup created in Task 4** was transferred to the new server, restored into /var/lib/jenkins, and Jenkins was started successfully.

The old Jenkins configuration was visible on the new server after restoration.

Conclusion

The Task 4 Jenkins backup was successfully restored onto a **fresh Jenkins server**. This demonstrated that the Jenkins environment can be recovered on another server using a Bash-generated backup, including the previously configured Jenkins data and configuration.